so-07-filesystem.pdf

VISIONE UTENTE

Semplificare l'esperienza dell'utente nell'accesso alle informazioni

Il Compito del **file system** è quello di astrarre la complessità di utilizzo dei diversi media proponendo una interfaccia per i sistemi di memorizzazione:

- comune
- efficiente
- conveniente da usare
  Dal punto di vista dell'utente, un file system è composto da due elementi:
- *file*: unità logica di memorizzazione
- *directory*: servono per organizzare e fornire informazioni sui file che compongono un file system
  Il concetto di file è l'entità atomica di assegnazione/gestione della memoria secondaria, è una collezione di informazioni correlate ⇒ fornisce una vista logica uniforme ad informazioni correlate

ATTRIBUTI DEI FILE ⇒ *NOME, TIPO, LOCAZIONE E DIMENSIONE, DATA E ORA*

- *informazioni sulla proprietà*: gruppi, utenti → accounting e autorizzazioni
- *attributi di protezione* → informazioni di accesso per verificare chi è autorizzato a eseguire operazioni sui file
- *flag*, informazioni di locking

TIPI DI FILE

A seconda della struttura interna

senza formato (stringa di byte): file testo,

con formato: file di record, file di database, a.out,

A seconda del contenuto

ASCII/binario (visualizzabile o no, 7/8 bit)

sorgente, oggetto, ...

eseguibile (oggetto attivo)

Alcuni S.O. supportano e riconoscono diversi tipi di file Conoscendo il tipo del file, il s.o. può evitare alcuni errori comuni, quali ad esempio stampare un file eseguibile

Esistono tre tecniche principali per identificare il tipo di un file

- meccanismo delle estensioni
- utilizzo di un attributo "tipo" associato al file nella directory

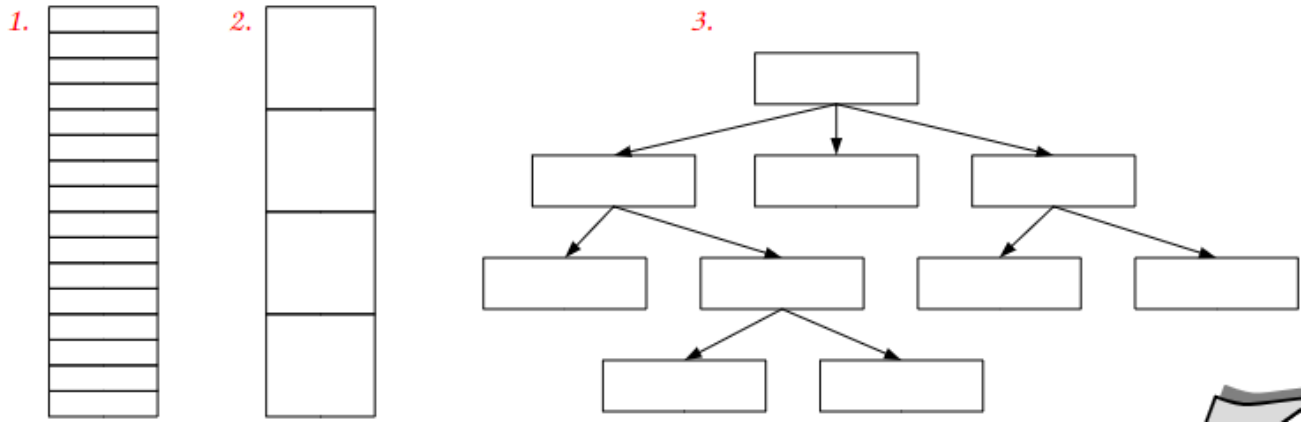
VIRTUALIZZAZIONE HARDWARE: ogni macchina VM ha un proprio kernel e sistema operativo completo, l'Hypervisor divide ogni VM dalle altre
le VM possono avere S.O. diversi più pesante richiede + memoria, CPU, sp
Simula CPU, RAM, disco, schede di rete ecc
es KVM
VMware

VIRTUALIZZ. SISTEMI OPER.: crea sistemi isolati (container) dentro lo stesso S.O. i container non hanno un proprio kernel, usano il kernel dell'host
Ogni container ha librerie, file e processi
tutti i container condividono lo stesso SO, più efficiente, veloce
es Docker

- magic number
  In un file system nei sistemi UNIX, sono presenti:
- file regolari ⟶ Sequenze di byte
- directory ⟶ file di sistema per mantenere la struttura del file system
- file speciali a blocchi ⟶ utilizzati per modellare dispositivi di I/O come i dischi (*)
- file speciali a caratteri ⟶ utilizzati per modellare device di I/O seriali come terminali, stampanti e reti (*)
- altri file speciali I ad es., pipe (*)
- (*) NON sono file!

I file possono essere strutturati in molti modi:

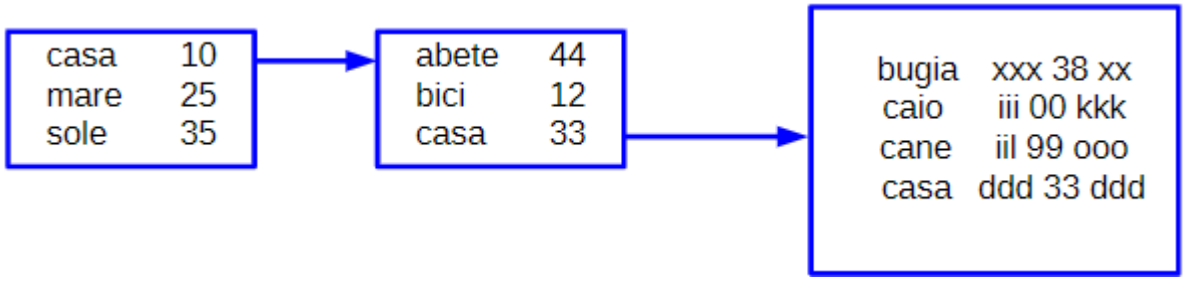
1. sequenze di byte
2. sequenze di record logici
3. file indicizzati (struttura ad albero)



I sistemi operativi possono attuare diverse scelte nella gestione della struttura dei file:
scelta minimale ⟹ i file sono considerati semplici stringhe di byte, a parte i file eseguibili il cui formato è dettato dal s.o.

Metodi di accesso
Sequenziale : read, write
Ad accesso diretto : read pos, write pos (oppure operazione seek)
Indicizzato : read key, write key ; Tipico dei database

Indice è una tabella di corrispondenza chiave-posizione
⟹ Può essere memorizzato in memoria: metodo efficiente ma dispendioso o su disco



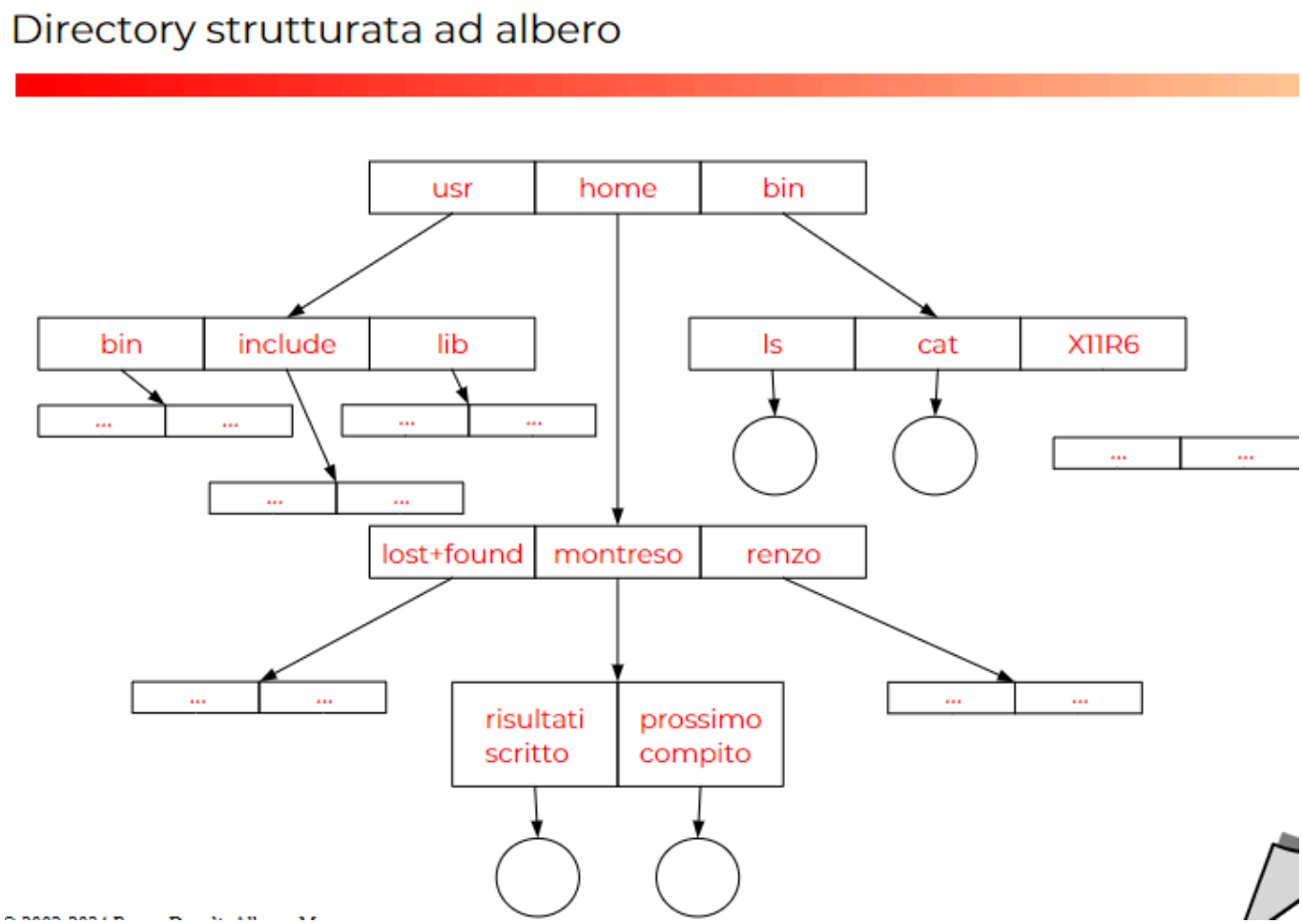
Operazioni fondamentali sui file
- creazione - apertura - chiusura - lettura - scrittura - append - posizionamento - cancellazione -troncamento -lettura/scrittura attributi

Operazioni sui file
L'API (interfaccia per la programmazione) relativa alle operazioni su file è basata sulle operazioni open/close ⇒ i file devono essere “aperti” prima di effettuare operazioni e “chiusi” al termine.
L’astrazione relativa all’apertura/chiusura dei file è utile per
mantenere le strutture dati di accesso al file
controllare le modalità di accesso e gestire gli accessi concorrenti
definire un descrittore per le operazioni di accesso ai dati

L'organizzazione dei file system è basata sul concetto di directory, che fornisce un'astrazione per un'insieme di file in molti sistemi, le directory sono file (speciali)

Operazioni definite sulle directory
creazione - cancellazione - apertura di una directory - chiusura di una directory - lettura di una directory - rinominazione - link/unlink

Struttura di una directory a livello singolo - a due livelli - ad albero - a grafo aciclico - a grafo



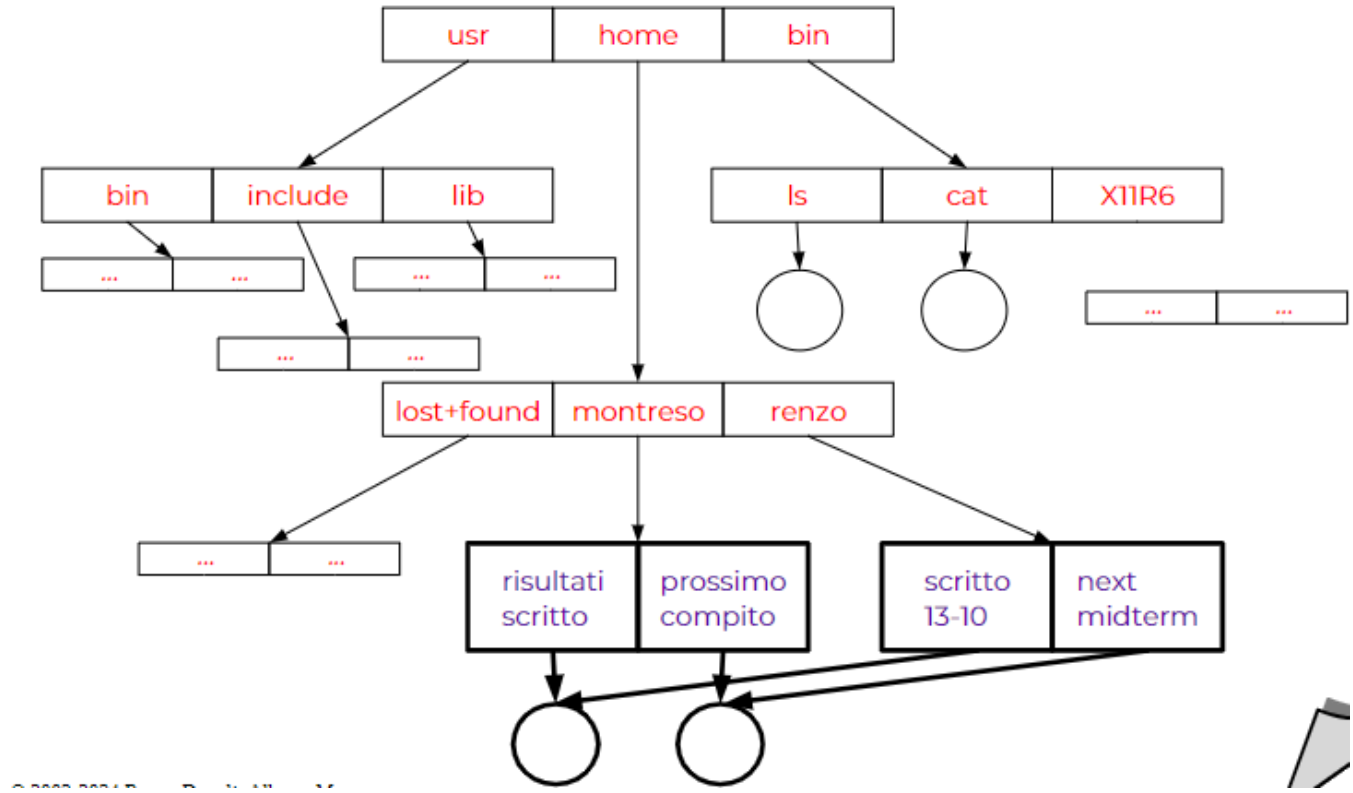
Directory strutturate a grafo aciclico

Nella struttura ad albero:

ogni file è contenuto in una directory univoca → E' anche possibile considerare grafi diversi dagli alberi : un file può essere contenuto in due o più directory

esiste un'unica copia del file suddetto: ogni modifica al file è visibile in entrambe le directory

La struttura risultante prende il nome di grafo diretto aciclico (DAG)



Semantica della coerenza

In un sistema operativo multitasking, i processi accedono ai file indipendentemente

Come vengono viste le modifiche ai file da parte dei vari processi?

In UNIX le modifiche al contenuto di un file aperto vengono rese visibili agli altri processi immediatamente.

esistono due tipi di condivisione del file:

- condivisione del puntatore alla posizione corrente nel file (condivisione ottenibile con fork)
- condivisione con distinti puntatori alla posizione corrente.

VISIONE IMPLEMENTATORE

Implementazione del file system ⇒ Problemi da tenere in considerazione, organizzazione di un disco, allocazione dello spazio in blocchi, gestione spazio libero, implementazione delle directory, tecniche per ottimizzare le prestazioni, tecniche per garantire la coerenza

Struttura di un disco

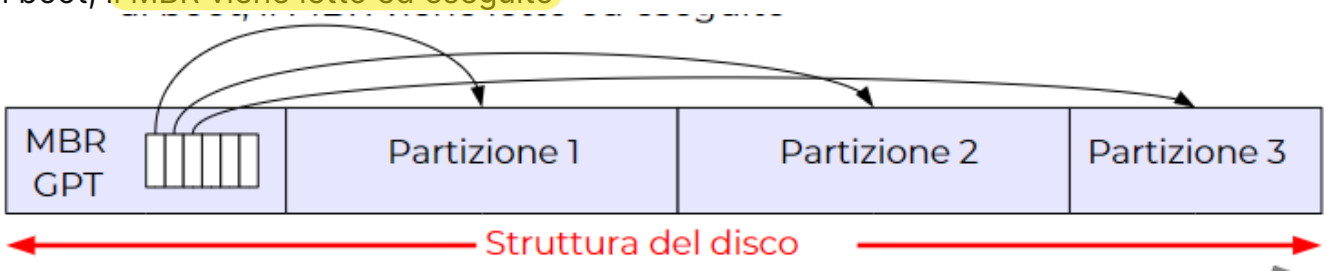
un disco può essere diviso in una o più partizioni, porzioni indipendenti del disco che possono ospitare file system distinti

il primo settore dei dischi è il cosiddetto master boot record (MBR)

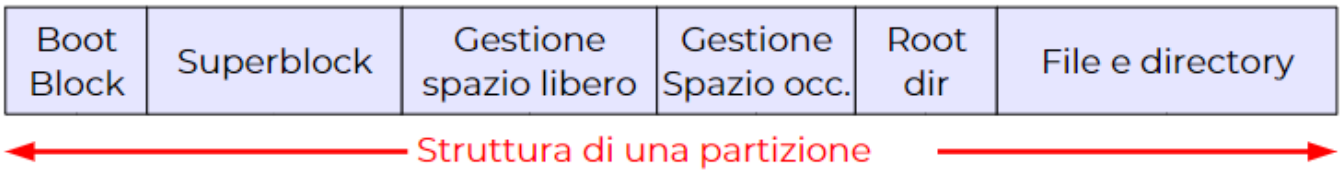
→è utilizzato per fare il boot del sistema

→contiene la partition table (tabella delle partizioni)

→contiene l'indicazione della partizione attiva
al boot, il MBR viene letto ed eseguito



Struttura di una partizione
ogni partizione inizia con un boot block
il MBR carica il boot block della partizione attiva e lo esegue
il boot block carica il sistema operativo e lo esegue
l'organizzazione del resto della partizione dipende dal file system



In generale
superblock: contiene informazioni sul tipo di file system e sui parametri fondamentali della sua organizzazione
tabelle per la gestione dello spazio libero: struttura dati contenente informazioni sui blocchi liberi
tabelle per la gestione dello spazio occupato: contiene informazioni sui file presenti nel sistema ⇒ non presente in tutti i file system
root dir: directory radice (del file system)
file e directory

Allocazione
Problema l'hardware e il driver del disco forniscono accesso al disco visto come un insieme di blocchi dati di dimensione fissa.
partendo da questa struttura come si implementa l'astrazione di file?
in altre parole: come vengono scelti i blocchi dati da utilizzare per un file e come questi blocchi dati vengono collegati assieme a formare una struttura unica
Questo è il problema dell'allocazione

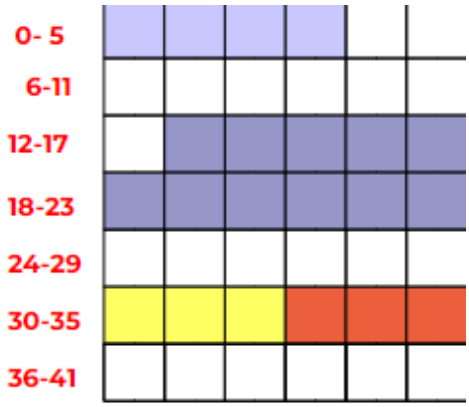
⇒ **Allocazione contigua**

Descrizione

- i file sono memorizzati in sequenze contigue di blocchi di dischi

Vantaggi

- non è necessario utilizzare strutture dati per collegare i blocchi
- l'accesso sequenziale è efficiente
  - blocchi contigui non necessitano operazioni di seek
- l'accesso diretto è efficiente
  - **block= offset/blocksize;**
  - **pos= offset%blocksize;**



directory

Name	Start	Size
a	0	4
b	13	11
c	30	3
d	33	3

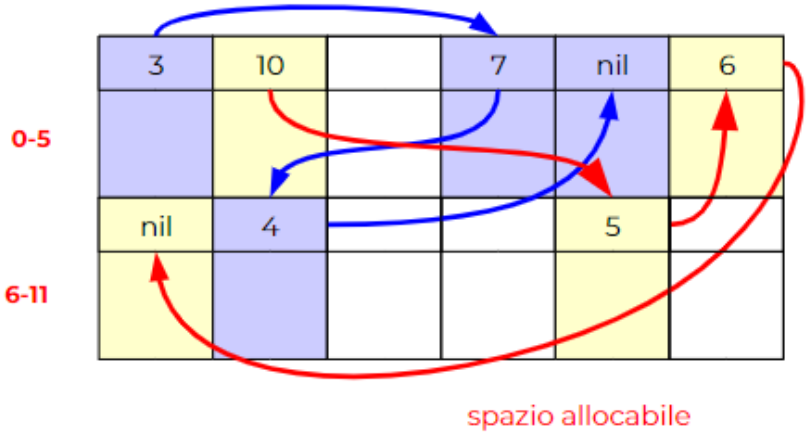
SVANTAGGI

si ripropongono tutte le problematiche dell'allocazione contigua in memoria centrale

- frammentazione esterna
- politica di scelta dell'area di blocchi liberi da usare per allocare spazio per un file: first fit, best fit, worst fit....
  inoltre: i file non possono crescere!

⇒ **ALLOCAZIONE CONTINUA**

Descrizione ogni file è costituito da un lista concatenata di blocchi.
ogni blocco contiene un puntatore al blocco successivo
il descrittore del file contiene i puntatori al primo e all'ultimo elemento della lista



directory

Name	Start	End
a	0	4
b	1	6

Vantaggi

- risolve il problema della frammentazione esterna
- l'accesso sequenziale o in "append mode" è efficiente

Svantaggi

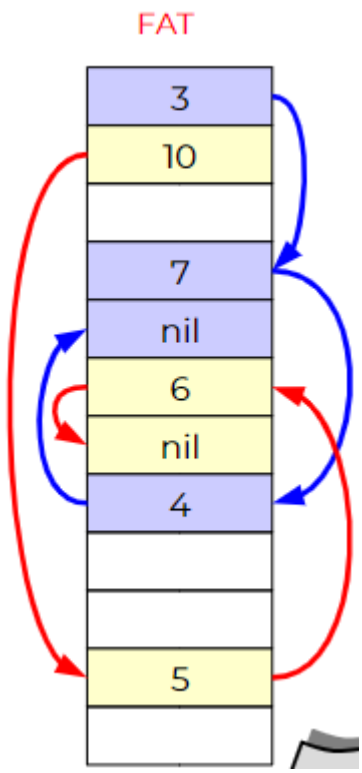
- l'accesso diretto è inefficiente
- progressivamente l'efficienza globale del file system degrada (i blocchi sono disseminati nel disco, aumenta il n. di seek)
- la dimensione utile di un blocco non è una potenza di due ⟹ se il blocco è piccolo (512 byte) l'overhead per i puntatori può essere rilevante

Minimizzare l'overhead dovuto ai puntatori ⇒ i blocchi vengono riuniti in cluster (contenenti 4, 8, 16 blocchi) e vengono allocati in modo indivisibile
Vantaggi la percentuale di spazio utilizzato per i puntatori diminuisce
Svantaggi aumenta lo spazio sprecato per la frammentazione interna

⟹ **ALLOCAZIONE BASATA SU FAT**
Descrizione invece di utilizzare parte del blocco dati per contenere il puntatore al blocco successivo si crea una tabella unica con un elemento per blocco (o per cluster)

directory

Name	Start	End
a	0	4
b	1	6



Vantaggi

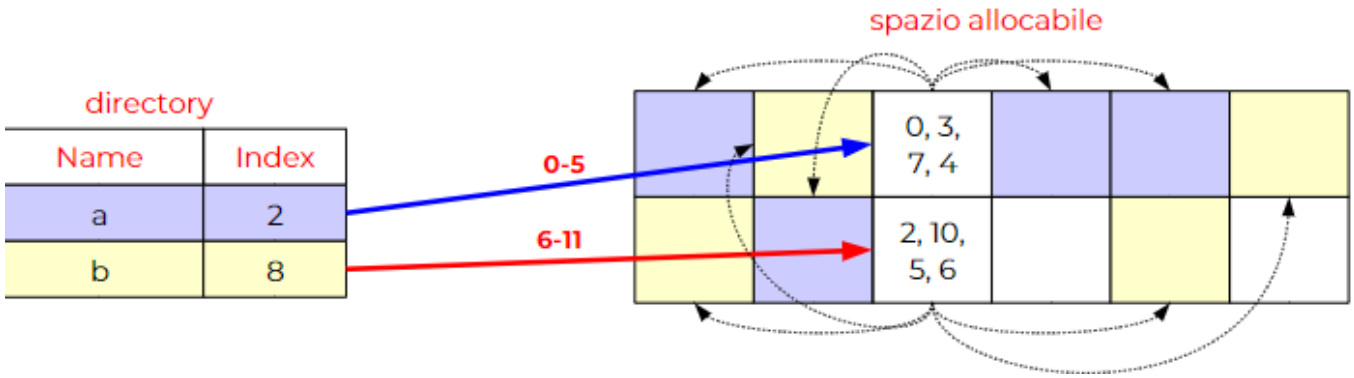
- i blocchi dati sono interamente dedicati... ai dati
- è possibile fare caching in memoria dei blocchi FAT
- l'accesso diretto diventa così più efficiente, in quanto la lista di puntatori può essere seguita in memoria

  Svantaggi
- la scansione richiede anche la lettura della FAT, aumentando così il numero di accessi al disco.

⇒ALLOCAZIONE INDICIZZATA

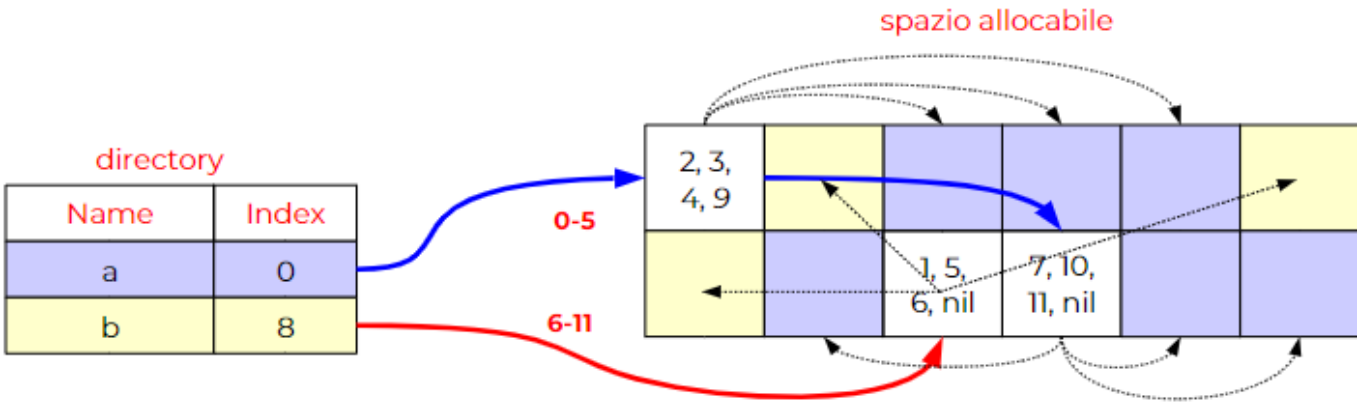
l'elenco dei blocchi che compongono un file viene memorizzato in un blocco (o area) indice

per accedere ad un file, si carica in memoria la sua area indice e si utilizzano i puntatori contenuti

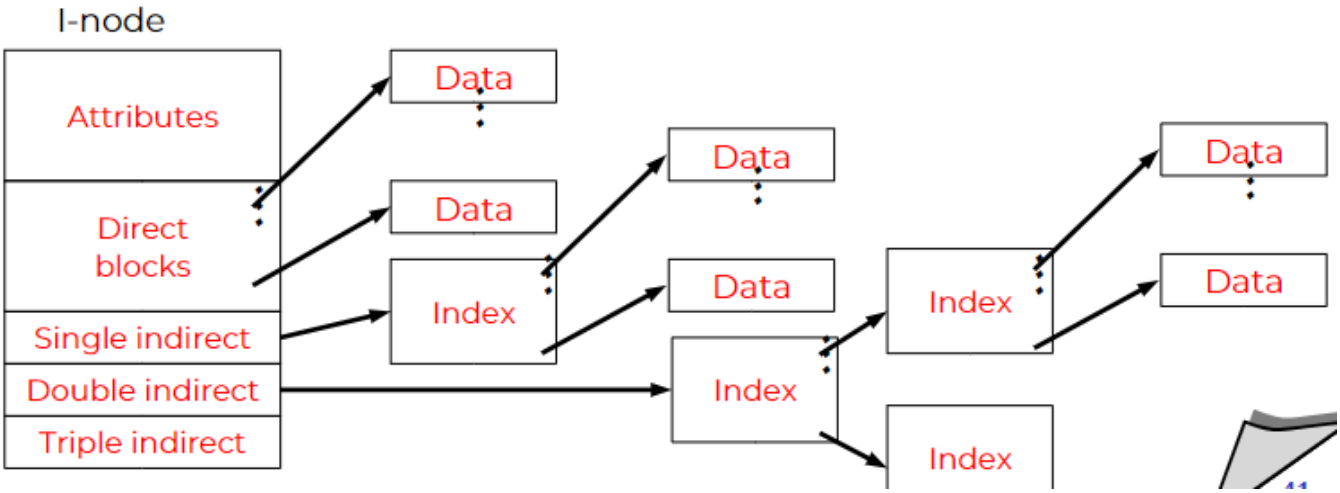


Vantaggi risolve (come l'allocazione concatenata) il problema della frammentazione esterna.
è efficiente per l'accesso diretto ⟹ il blocco indice deve essere caricato in memoria solo quando il file è aperto
Svantaggi la dimensione del blocco indice determina l'ampiezza massima del file ⟹ utilizzare blocchi indici troppo grandi comporta un notevole spreco di spazio

Concatenazione di blocchi indice l'ultimo elemento del blocco indice non punta al blocco dati ma al blocco indice successivo; si ripropone il problema per l'accesso diretto a file di grandi dimensioni
possibili soluzioni



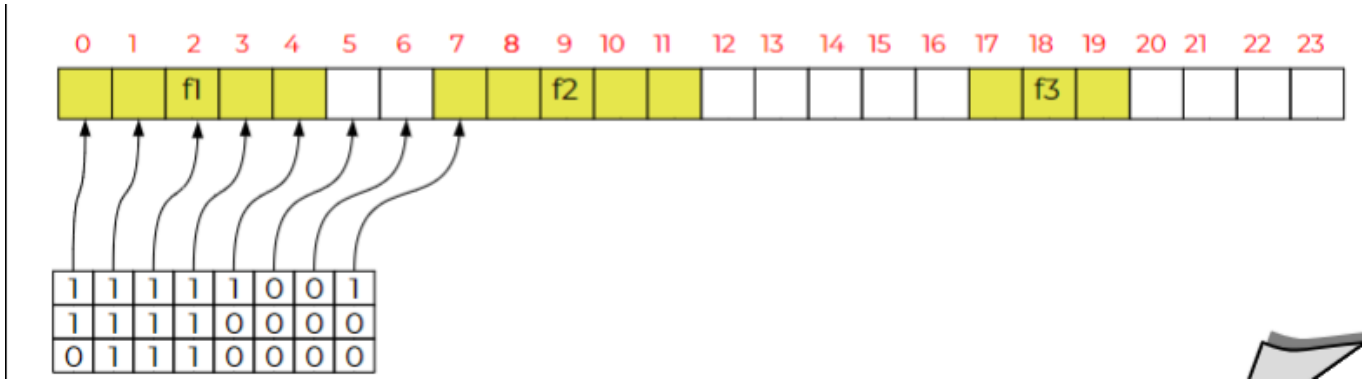
In UNIX
ogni file è associato ad un i-node (index node)
un i-node è una struttura dati contenente gli attributi del file, e un indice di blocchi diretti e indiretti, secondo uno schema misto



Lo schema UNIX
garantisce buone performance nel caso di accesso sequenziale
file brevi sono acceduti più velocemente e occupano meno memoria

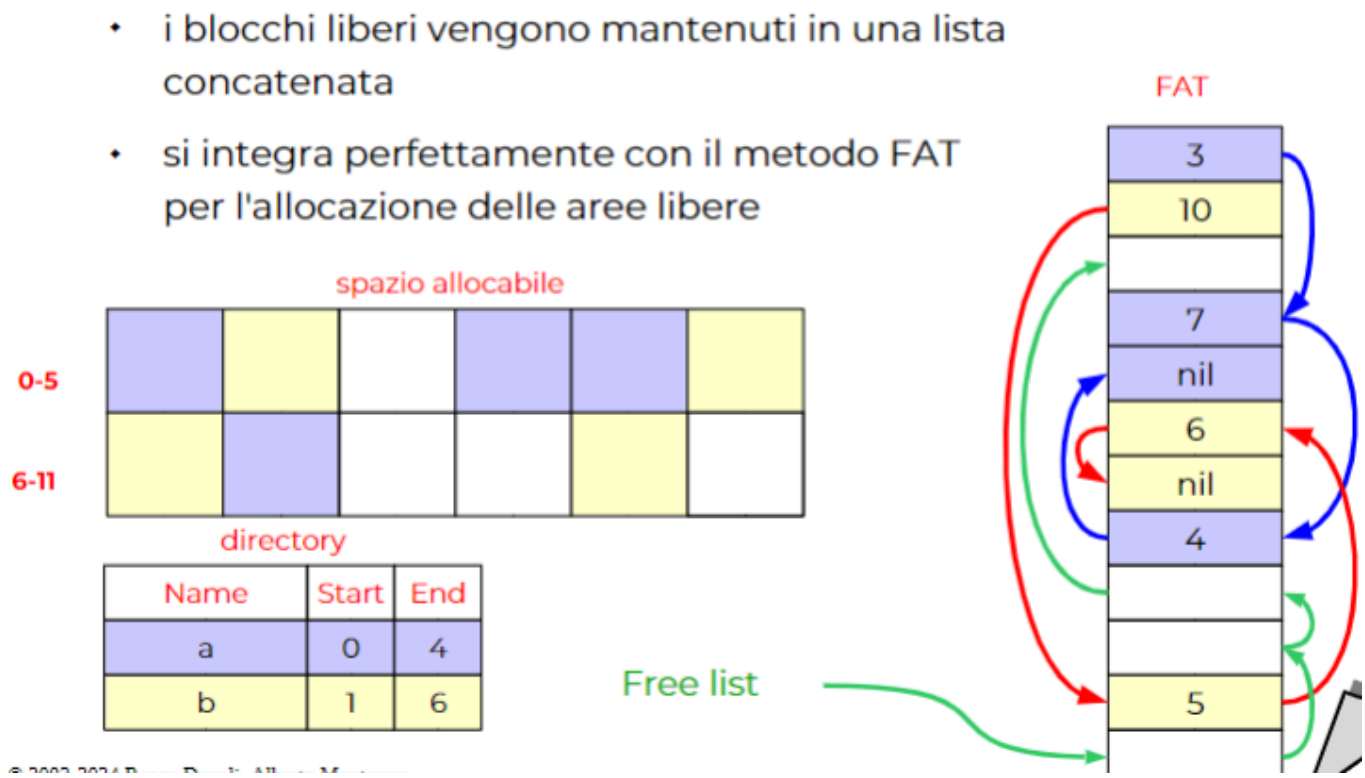
pre-caricamento (per esempio nell'allocazione concatenata fornisce buone prestazioni per l'accesso sequenziale).
combinazione dell'allocazione contigua e indicizzata
contigua per piccoli file ove possibile
indicizzata per grandi file
Indicizzata di chunk (sotto-sequenze) contigue (ext4)

Mappa di bit
ad ogni blocco corrisponde un bit in una bitmap
i blocchi liberi sono associati ad un bit di valore 0, i blocchi occupati sono associati ad un bit di valore 1



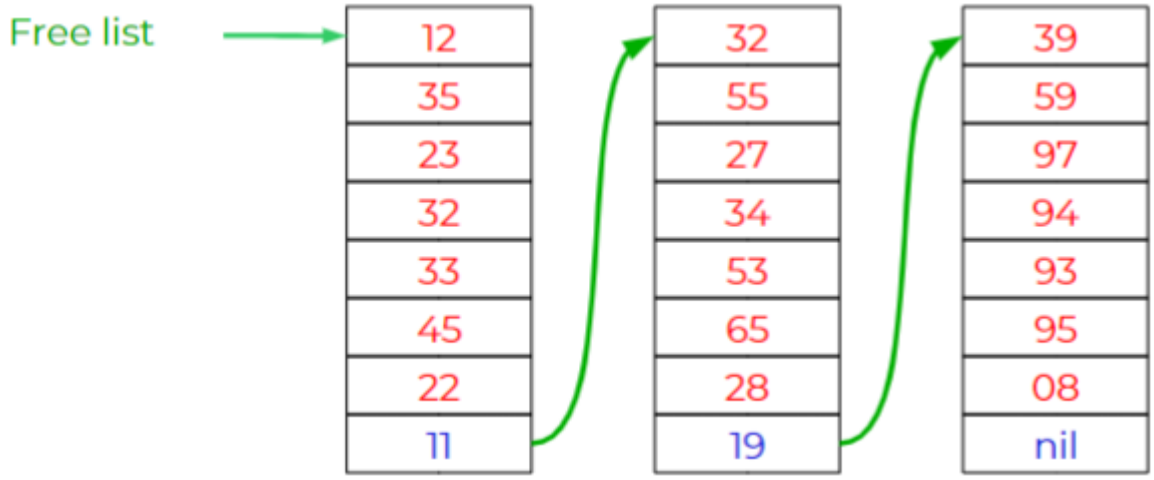
Vantaggi semplice, è possibile selezionare aree contigue
Svantaggi la memorizzazione del vettore può richiedere molto spazio

Lista concatenata



Vantaggi richiede poco spazio in memoria centrale
Svantaggi l'allocazione di un'area di ampie dimensioni è costosa ; l'allocazione di aree libere contigue è molto difficoltosa

Lista concatenata (blocchi)
è costituita da una lista concatenata di blocchi contenenti puntatori a blocchi liberi



Vantaggi ad ogni istante, è sufficiente mantenere in memoria semplicemente un blocco contenente elementi liberi
non è necessario utilizzare una struttura a dati a parte; i blocchi contenenti elenchi di blocchi liberi possono essere mantenuti all'interno dei blocchi liberi stessi
Svantaggi l'allocazione di un'area di ampie dimensioni è costosa ⟹ l'allocazione di aree libere contigue è molto difficoltosa

Implementazione delle directory
Una directory è un file speciale contenente informazioni sui file contenuti nella directory
→una directory è suddivisa in un certo numero di *directory entry* ⟹ *ogni directory entry* deve permettere di accedere a tutte le informazioni necessarie per gestire il file

- nome
- attributi
- informazioni di allocazione
  Scelte implementative da considerare
  attributi contenuti nelle directory entry oppure nell'i-node
  lista lineare (array) vs hash table

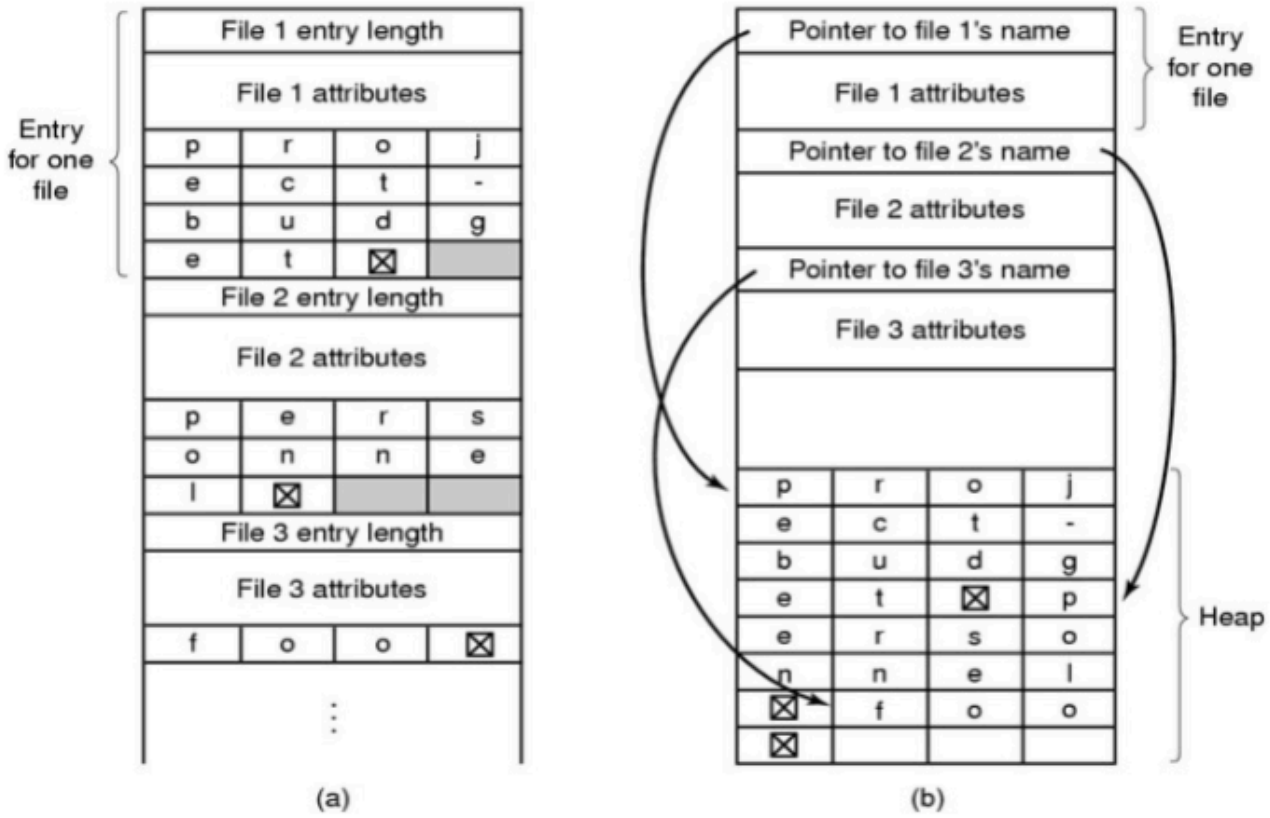
Informazioni nelle directory entry ⟹ la directory entry contiene tutte le informazioni necessarie associate ad un file
Informazione negli i-node ⟹ le informazioni sono contenute negli i-node; una directory entry contiene un indice di i-node I utilizzata in UNIX

nome file
attributi
info allocaz.
nome file
attributi
info allocaz.

nome	i-node
nome	i-node
nome	i-node
nome	i-node

Problema della lunghezza dei nomi ⟹ memorizzare nomi a lunghezza variabile nelle directory comporta una serie di problemi
Lunghezza fissa ⟹ è il meccanismo più semplice... ma presenta un trade-off:

- spazio riservato molto grande, spreco di memoria
- spazio riservato troppo piccolo, nomi troppo brevi
  Lunghezza variabile ⟹ la struttura dati diviene più complessa ; due possibili esempi nel lucido successivo



Lista lineare ⟹

- semplice da implementare
- inefficiente nel caso di directory di grandi dimensioni
  Tabella hash ⟹
- occorre stabilire la dimensione della tabella di hash e il metodo di gestione delle collisioni
- la gestione può essere inefficiente se ci sono numerose collisioni

Directory strutturata a grafo aciclico

Due implementazioni possibili

link simbolici e hard link

Link simbolici ⟹ viene creato un tipo speciale di directory entry, che contiene un riferimento (sotto forma di cammino assoluto) al file in questione

quando viene fatto un riferimento al file ⟶ si cerca nella directory → si scopre che si tratta di un link ; viene risolto il link (ovvero, viene utilizzato il cammino assoluto registrato nel file

Hard link ⟹ le informazioni relative al file sono presenti, uguali, in entrambe le directory -→ non è necessario una doppia ricerca nel file system; è impossibile distinguere la copia dall'originale

Considerazioni

una struttura a grafo diretto aciclico è più flessibile di un albero, ma crea tutta una serie di problemi nuovi

non tutti i file system sono basati su DAG; esempio MS-DOS non li supporta